

Employee Payroll Management System

J1.L.P0037 | UML, Algorithms, and Test Evidence

Document field	Value
Student	Nguyen Dinh Chuong (Nguyễn Đình Chương)
Student ID	SE204330
Course	LAB211 - OOP with Java Lab
Release	Portfolio review v1.0 - 21 July 2026
Repository	https://github.com/KiritoMainBro88/SU26-LAB211

Scope intentionally limited to the lecturer's review priorities: final UML, implemented algorithms, test cases, and reproducible evidence.

1. Executive Summary

A Java console payroll application that validates seven Employee fields, tolerates bad rows without accepting invalid state, performs lecturer-specified soft deletion, searches by four attributes, and calculates payroll for active Employees only.

1.1 Verification snapshot

Check	Result	Evidence
Java compatibility compile	PASS	javac targeting Java 8, UTF-8, and -Xlint:all
Executable deep verification	57 passed, 0 failed	labs/lab3/testcases/Lab3DeepVerification.java
Import audit	PASS	0 wildcard, 0 unresolved, 0 definitely unused explicit imports
Report/Audit metadata	ALIGNED	J1.L.P0037, SE204330, 21 July 2026
Required functions traced	9/9	Section 5 traceability matrix

1.2 Evidence interpretation

The runnable deep harness was recompiled and executed during this review: 57 checks passed and none failed. The smaller verification_results.txt file is retained as earlier regression evidence (17 passed), but is not counted in the current total.

2. UML Design

This diagram is regenerated from the final source structure and names only implemented classes, interfaces, and relationships relevant to grading and maintenance.

J1.L.P0037 - Final UML Class Relationships

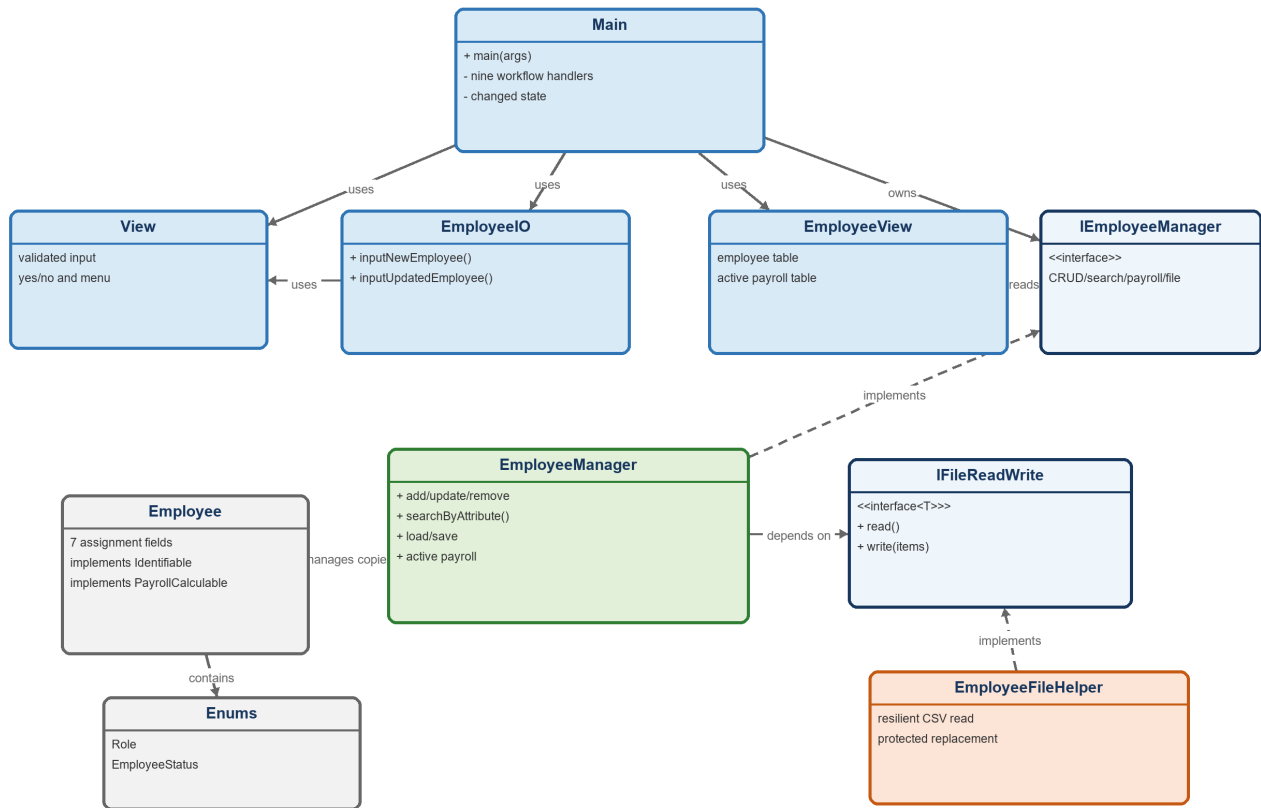


Figure 1. Final class relationships for J1.L.P0037.

2.1 Responsibility matrix

Class / Interface	Final responsibility
Main	Coordinates the nine workflows and changed-state save/quit behavior.
View / EmployeeView	Own reusable validated input and employee/payroll table presentation.
EmployeeIO	Builds validated add/update candidates without business-state ownership.
IEmployeeManager / EmployeeManager	Own CRUD, soft delete, search, payroll, defensive copies, load, and save.
IFileReadWrite<T>	Generic persistence boundary injected into EmployeeManager.
EmployeeFileHelper	Reads resilient CSV rows and performs validated temporary-file replacement.
Employee	Implements Identifiable and PayrollCalculable for domain-focused polymorphism.
Role / EmployeeStatus	Constrain textual values and centralize canonical display/parse behavior.

2.2 OOP evidence

- Abstraction: IEmployeeManager and IFileReadWrite<T> separate business and persistence contracts.
- Polymorphism: Employee implements Identifiable and PayrollCalculable behaviors.
- Dependency inversion: EmployeeManager receives its file helper through constructor injection.

3. Implemented Algorithms

Pseudocode reflects the final implementation rather than an idealized alternative. Complexity statements account for the list-based structures used by these assignment-sized applications.

Algorithm Flow - Resilient Employee Load

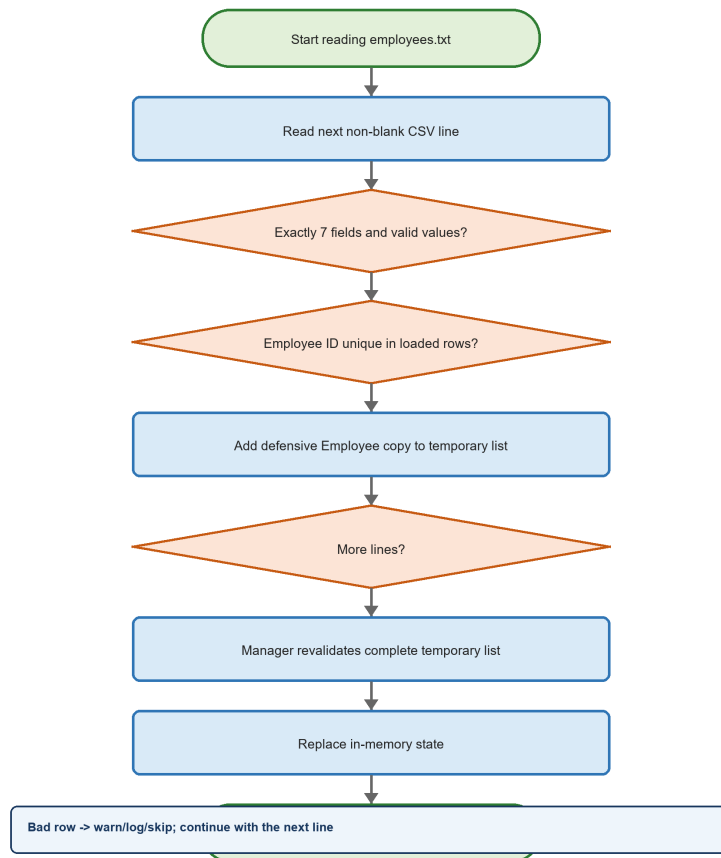


Figure 2. Representative decision flow from the final implementation.

A1 - Row-isolated employee loading

Purpose: Continue after erroneous lines while never admitting malformed or duplicate state.

```
01 if employees.txt is missing: fail the load
02 for each non-blank physical line:
03     parse exactly seven CSV fields
04     parse Role/Status and finite numeric fields
05     validate complete Employee and duplicate ID
06     on row error: warn, log, skip, and continue
07 manager validates the completed list again, then replaces memory
```

Complexity: $O(n^2)$ time from list-based duplicate checks and $O(n)$ result space.

A2 - Validate-before-commit update

Purpose: Update only Role, Base Salary, Bonus, and Status as required by Function 3.

```
01 find current Employee by immutable ID
02 construct candidate with old ID, Name, and Working Days
03 apply only the four permitted candidate fields
04 validate the complete candidate
05 commit four fields only after validation succeeds
```

Complexity: $O(n)$ lookup time and $O(1)$ additional space.

A3 - Soft delete and active-only payroll

Purpose: Retain history while excluding inactive Employees from payroll.

```
01 REMOVE: find Employee; reject missing or already inactive record
02 REMOVE: set status to INACTIVE; keep list size unchanged
03 PAYROLL: iterate Employees and include only status == ACTIVE
04 salary = baseSalary * workingDays + bonus
05 sum and display the same active set
```

Complexity: $O(n)$ time and $O(a)$ output space for a active Employees.

A4 - Safe employee-file save

Purpose: Never truncate the previous target before validation and complete writing.

```
01 validate list, every Employee, and unique IDs
02 create a temporary sibling beside the target
03 write escaped UTF-8 CSV with decimal-safe number formatting
04 replace target atomically when supported
05 on failure delete temporary data and preserve previous target
```

Complexity: $O(n^2)$ validation time and $O(1)$ streaming write space.

4. Test Cases and Executed Evidence

4.1 Execution record

Artifact	Status	Interpretation
Source compilation	PASS	All production and deep-test Java files compile for Java 8.
Deep harness	57/57 PASS	Re-executed 21 July 2026; exit code 0.
Earlier regression record	17/17 PASS	Historical result; not added to the current 153-check claim.
State preservation	PASS	Failure paths assert memory/file content, not only return values.

4.2 Representative test matrix

ID	Scenario	Input / action	Expected result	Actual
L3-T01	Dependency/null guards	Null helper, View, EmployeeIO dependencies	IllegalArgumentException	PASS
L3-T02	Working-day boundaries	0, 26, and 27	0/26 accepted; 27 rejected	PASS
L3-T03	Non-finite salary	Infinity salary; negative bonus	Rejected	PASS
L3-T04	Restricted update	Candidate changes all fields	Name/days preserved; four fields changed	PASS
L3-T05	Soft delete	Remove active E003 twice	Inactive record retained; second remove rejected	PASS
L3-T06	Exact status search	active with active/inactive data	Inactive does not match active	PASS
L3-T07	Malformed-row isolation	valid, malformed, duplicate, valid	Two valid records loaded	PASS
L3-T08	CSV quoting/decimals	Doe, Jane; 120.5; 3.25	Round-trip preserves values	PASS
L3-T09	Rejected save preservation	NaN bonus after valid target exists	Write rejected; target unchanged	PASS
L3-T10	Polymorphic contracts	Employee through both interfaces	Identity/payroll behavior matches	PASS

4.3 Coverage strategy

- Equivalence partitions: valid, invalid format, duplicate, missing reference, and empty/null inputs.
- Numeric boundaries: lower/upper limits plus NaN and positive infinity for double values.
- State assertions: rejected operations preserve the prior object list or target file.
- Persistence round-trips: valid object/text data reloads with essential values preserved.
- Encapsulation checks: mutations to returned objects/lists cannot alter managed state.

5. Requirement-to-Source Traceability

Every visible assignment function maps to a final handler, implementation method, and executable or directly inspectable verification artifact.

Requirement	Menu / handler	Final implementation	Evidence
1. Load	Main.manualLoadData	EmployeeManager.loadFromFile	L3-T07; failure preservation
2. Add	Main.addNewEmployee	EmployeeManager.addEmployee	L3-T01-T03
3. Update	Main.updateEmployee	EmployeeManager.updateEmployee	L3-T04
4. Remove (D_Soft)	Main.removeEmployee	EmployeeManager.removeEmployee	L3-T05
5. Search	Main.searchEmployee	EmployeeManager.searchByAttribute	L3-T06
6. Payroll	Main.calculatePayroll	EmployeeManager.getTotalSalaryByMonth	Active-only checks
7. Display	Main case 7	EmployeeView.displayAllEmployees	Empty/long-name output
8. Save	Main.saveData	EmployeeFileHelper.write	L3-T08, T09
9. Quit	Main.quitProgram	Changed-state save decision	Source branch trace

5.1 Import and source hygiene

Audit item	Result	Basis
Incorrect/unresolved imports	0	Complete production + test compilation succeeded.
Wildcard imports	0	Repository-wide source scan.
Definitely unused explicit imports	0	Imported simple names are referenced in their units.
Generated binaries committed	0	.gitignore excludes build, dist, class, jar, private IDE state, and logs.

5.2 Known limitations and deliberate trade-offs

- A file containing only invalid non-blank rows produces an empty valid result after warnings; this recovery policy is documented.
- Soft delete follows the lecturer's D_Soft note even though the assignment PDF uses the shorter word 'Delete'.
- List scans favor clarity for assignment-sized data; indexed structures would be preferable at enterprise scale.

5.3 Reproduction

From the repository root on Windows:

```
PowerShell -ExecutionPolicy Bypass -File .\scripts\verify-all.ps1
```

GitHub Actions invokes the same script on every push and pull request. The workflow pins actions to reviewed commit SHAs and grants read-only repository permission.

Conclusion

The final J1.L.P0037 source implements all 9 required functions, compiles for Java 8, and passes 57 executable deep checks. The UML, algorithms, test evidence, and AI Audit Log use the same identifiers, counts, paths, and verification date.